Lab.4

UWAGA: *Zadania zostały połączone w 5 programów (zgodnie z dozwolonym łączeniem). Zadania 1-8 wykorzystują kolejki komunikatów System V (msgget, msgsnd, msgrcv), które naturalnie buforują dane, zachowują kolejność i są synchronizowane przez jądro. Zadanie 9 wykorzystuje pamięć współdzieloną z semaforami, gdzie synchronizację trzeba zapewnić ręcznie.*

Zadanie 1-2

Program tworzy przez fork() dwa procesy: producenta i konsumenta. Oba wypisują swój PID (getpid()). Komunikacja odbywa się przez wspólną kolejkę komunikatów utworzoną funkcją msgget().

Producent wysyła jeden komunikat tekstowy funkcją msgsnd(), a konsument odbiera go przez msgrcv() i wypisuje na ekranie. Realizowane wspólnie z Zadaniem 1 w programie lab4_z1_2.c.

```
root@debian:~# gcc lab4_z1_2.c -o lab4_z1_2
root@debian:~# ./lab4_z1_2
Konsument PID 840: czekam
Producent PID 841: wysylam
Konsument odebral: Dane od producenta
root@debian:~#
```

```c
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
#include <sys/ipc.h>
#include <sys/msg.h>

struct mymsg {
long mtype;
char mtext[100];
};

int main() {
key_t key = 1234;
int msgid = msgget(key, 0666 | IPC_CREAT);
pid_t pid = fork();

if (pid == 0) {
struct mymsg msg;
msg.mtype = 1;
strcpy(msg.mtext, "Dane od producenta");
printf("Producent PID %d: wysylam\n", getpid());
msgsnd(msgid, &msg, sizeof(msg.mtext), 0);
} else {
struct mymsg msg;
printf("Konsument PID %d: czekam\n", getpid());
msgrcv(msgid, &msg, sizeof(msg.mtext), 1, 0);
printf("Konsument odebral: %s\n", msg.mtext);
wait(NULL);
msgctl(msgid, IPC_RMID, NULL);
}
return 0;
}
```

Zadanie 3-4

Producent wysyła serię liczb (od 5 do 10 sztuk) jako osobne komunikaty. Konsument odczytuje je przez msgrcv() i wypisuje w tej samej kolejności, ponieważ kolejka działa w trybie FIFO (pierwsze wysłane, pierwsze odebrane).

Po odebraniu wszystkich liczb konsument odsyła producentowi komunikat potwierdzający. Wykorzystano pole mtype: producent wysyła z typem 1, a potwierdzenie wraca z typem 2, dzięki czemu jedna kolejka obsługuje komunikację w obie strony. Realizowane wspólnie z Zadaniem 3 w programie lab4_z3_4.c.

```
root@debian:~# gcc -std=gnu99 lab4_z3_4.c -o lab4_z3_4
root@debian:~# ./lab4_z3_4
Konsument odebral: 1
Producent wyslal: 1
Konsument odebral: 2
Producent wyslal: 2
Konsument odebral: 3
Producent wyslal: 3
Konsument odebral: 4
Producent wyslal: 4
Konsument odebral: 5
Producent wyslal: 5
Konsument odebral: 6
Producent wyslal: 6
Konsument odebral: 7
Producent wyslal: 7
Producent: odebral potwierdzenie: OK wszystko odebrane
root@debian:~#
```

```
 GNU nano 7.2
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
#include <sys/ipc.h>
#include <sys/msg.h>

struct mymsg {
long mtype;
char mtext[100];
};

int main() {
int msgid = msgget(1235, 0666 | IPC_CREAT);
pid_t pid = fork();

if (pid == 0) {
struct mymsg msg;
for (int i = 1; i <= 7; i++) {
msg.mtype = 1;
sprintf(msg.mtext, "%d", i);
msgsnd(msgid, &msg, sizeof(msg.mtext), 0);
printf("Producent wyslal: %d\n", i);
}
msgrcv(msgid, &msg, sizeof(msg.mtext), 2, 0);
printf("Producent: odebral potwierdzenie: %s\n", msg.mtext);
} else {
struct mymsg msg;
for (int i = 0; i < 7; i++) {
msgrcv(msgid, &msg, sizeof(msg.mtext), 1, 0);
printf("Konsument odebral: %s\n", msg.mtext);
}
msg.mtype = 2;
strcpy(msg.mtext, "OK wszystko odebrane");
msgsnd(msgid, &msg, sizeof(msg.mtext), 0);
wait(NULL);
msgctl(msgid, IPC_RMID, NULL);
}
return 0;
}_
```

Zadanie 5-6

Dostęp do wspólnych danych jest zabezpieczony, ponieważ operacje na kolejce komunikatów (msgsnd, msgrcv) są atomowe i synchronizowane przez jądro systemu. Kilka procesów może jednocześnie pisać i czytać bez mieszania się danych ani konfliktu.

Program uruchamia dodatkowego producenta i konsumenta (łącznie po dwa). Wszystkie korzystają z tej samej kolejki. Widać, że komunikacja działa poprawnie przy większej liczbie procesów, a komunikaty nie giną. Realizowane wspólnie z Zadaniem 5 w programie lab4_z5_6.c.

```
GNU nano 7.2
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
#include <sys/ipc.h>
#include <sys/msg.h>

struct mymsg {
long mtype;
char mtext[100];
};

int main() {
int msgid = msgget(1236, 0666 | IPC_CREAT);

for (int p = 1; p <= 2; p++) {
if (fork() == 0) {
struct mymsg msg;
msg.mtype = 1;
sprintf(msg.mtext, "Producent %d (PID %d)", p, getpid());
msgsnd(msgid, &msg, sizeof(msg.mtext), 0);
return 0;
      }
}

for (int c = 1; c <= 2; c++) {
if (fork() == 0) {
struct mymsg msg;
msgrcv(msgid, &msg, sizeof(msg.mtext), 1, 0);
printf("Konsument %d (PID %d) odebral: %s\n", c, getpid(), msg.mtext);
return 0;
   }
}
for (int i = 0; i < 4; i++)
wait(NULL);
printf("Rodzic: wszystkie procesy skonczone\n");
msgctl(msgid, IPC_RMID, NULL);
return 0;
}
```

```
root@debian:~# gcc -std=gnu99 lab4_z5_6.c -o lab4_z5_6
root@debian:~# ./lab4_z5_6
Konsument 1 (PID 936) odebral: Producent 2 (PID 935)
Konsument 2 (PID 937) odebral: Producent 1 (PID 934)
Rodzic: wszystkie procesy skonczone
root@debian:~#
```

Zadanie 7-8

Wprowadzono limit liczby komunikatów w kolejce. Gdy kolejka jest pełna, producent czeka (próba wysłania z flagą IPC_NOWAIT zwraca błąd EAGAIN, więc producent ponawia próbę). Konsument po pobraniu komunikatu zwalnia miejsce i odblokowuje producenta.

Dodano mechanizm sygnałów: producent po przygotowaniu nowego zadania wysyła sygnał SIGUSR1 do konsumenta funkcją kill(), powiadamiając go, że nowe dane czekają w kolejce. Realizowane wspólnie z Zadaniem 7 w programie lab4_z7_8.c.

```
root@debian:~# ./lab4_z7_8
Producent wyslal: zadanie 1
Producent wyslal: zadanie 2
Producent wyslal: zadanie 3
Producent: kolejka pelna, czekam...
Producent: kolejka pelna, czekam...
Konsument odebral: zadanie 1
Producent wyslal: zadanie 4
Producent: kolejka pelna, czekam...
Producent: kolejka pelna, czekam...
Producent: kolejka pelna, czekam...
Konsument odebral: zadanie 2
Producent wyslal: zadanie 5
Producent: kolejka pelna, czekam...
Producent: kolejka pelna, czekam...
Producent: kolejka pelna, czekam...
Konsument odebral: zadanie 3
Producent wyslal: zadanie 6
Konsument odebral: zadanie 4
Konsument odebral: zadanie 5
root@debian:~# _
```

```
  GNU nano 7.2
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <signal.h>
#include <errno.h>
#include <sys/wait.h>
#include <sys/ipc.h>
#include <sys/msg.h>

#define LIMIT 3

struct mymsg {
long mtype;
char mtext[100];
};

void handler(int sig) {
write(1, "Konsument: nowe zadanie gotowe!\n", 32);
}

int main() {
int msgid = msgget(1237, 0666 | IPC_CREAT);
pid_t pid = fork();

if (pid == 0) {
struct sigaction sa;
sa.sa_handler = handler;
sigemptyset(&sa.sa_mask);
sa.sa_flags = 0;
sigaction(SIGUSR1, &sa, NULL);

for (int i = 0; i < 5; i++) {
sleep(2);
struct mymsg msg;
msgrcv(msgid, &msg, sizeof(msg.mtext), 1, 0);
printf("Konsument odebral: %s\n", msg.mtext);
sleep(1);
  }
} else {
int sent = 0;
for (int i = 0; i <= 5; i++) {
struct mymsg msg;
msg.mtype = 1;
sprintf(msg.mtext, "zadanie %d", i + 1);
while (sent >= LIMIT) {
printf("Producent: kolejka pelna, czekam...\n");
sleep(1);
struct msqid_ds info;
msgctl(msgid, IPC_STAT, &info);
sent = info.msg_qnum;
     }

msgsnd(msgid, &msg, sizeof(msg.mtext), 0);
sent++;
printf("Producent wyslal: %s\n", msg.mtext);
}
wait(NULL);
msgctl(msgid, IPC_RMID, NULL);
}
return 0;
}
_

^G Help          ^O Write Out     ^W Where Is      ^
^X Exit          ^R Read File     ^\ Replace       ^
```

Zadanie 9

Wspólna kolejka znajduje się w pamięci współdzielonej (mmap). Dostęp do niej kontrolują semafory nienazwane (sem_init, sem_wait, sem_post): jeden chroni sekcję krytyczną (wzajemne wykluczanie), dodatkowe liczą wolne i zajęte miejsca. Dzięki temu producent i konsument nie wchodzą w konflikt przy dostępie do wspólnej kolejki.

```
 GNU nano 7.2                                                    lab4_z9.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <semaphore.h>
#include <sys/mman.h>
#include <sys/wait.h>

#define MAX 5

typedef struct {
int dane[MAX];
int head;
int tail;
int count;
sem_t mutex;
sem_t wolne;
sem_t zajete;
} Kolejka;

int main() {
Kolejka *q = mmap(NULL, sizeof(Kolejka), PROT_READ | PROT_WRITE, MAP_SHARED | MAP_ANONYMOUS, -1, 0);

q->head = q->tail = q->count = 0;
sem_init(&q->mutex, 1, 1);
sem_init(&q->wolne, 1, MAX);
sem_init(&q->zajete, 1, 0);

if (fork() == 0) {
for (int i = 1; i <= 5; i++) {
sem_wait(&q->wolne);
sem_wait(&q->mutex);
q->dane[q->tail] = i;
q->tail = (q->tail + 1) % MAX;
q->count++;
printf("Producent dodal: %d\n", i);
sem_post(&q->mutex);
sem_post(&q->zajete);
sleep(1);
    }
} else {
for (int i = 0; i < 5; i++) {
sem_wait(&q->zajete);
sem_wait(&q->mutex);
int val = q->dane[q->head];
q->head = (q->head + 1) % MAX;
q->count--;
printf("Konsument pobral: %d\n", val);
sem_post(&q->mutex);
sem_post(&q->wolne);
sleep(2);
}
wait(NULL);
sem_destroy(&q->mutex);
sem_destroy(&q->wolne);
sem_destroy(&q->zajete);
munmap(q, sizeof(Kolejka));
      }
return 0;
}

^G Help          ^O Write Out     ^W Wher
^X Exit          ^R Read File     ^\ Repl
```

```
root@debian:~# gcc lab4_z9.c -o lab4_z9 -pthread
root@debian:~# ./lab4_z9
Producent dodal: 1
Konsument pobral: 1
Producent dodal: 2
Konsument pobral: 2
Producent dodal: 3
Producent dodal: 4
Konsument pobral: 3
Producent dodal: 5
Konsument pobral: 4
Konsument pobral: 5
root@debian:~#
```